User-Driven Product Analysis via Web Scraping & Multi-modal NLP: Sentiment Analysis, Feature Extraction, and Recommendation using Amazon Reviews

A GRADUATION PROJECT SUBMITTED TO
THE FACULTY OF ENGINEERING AND ARCHITECTURE OF
UNIVERSITY OF NEW YORK TIRANA
IN PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE DEGREE OF
BACHELOR OF SCIENCE IN COMPUTER SCIENCE

Gerind Tershana
June 2024

UNYT

APPROVAL

This is to certify that I have read this project and that, in my opinion, it is fully adequate, in scope and quality, as a thesis for the degree of Bachelor of Science in Computer Science.

(Title and Name)
(Project Advisor) Dr. Marjana Prifti Skenduli

This is to confirm that this thesis complies with all the standards set by the Department of Computer Science at the Faculty of Engineering and Architecture at the University of New York Tirana.

Date Seal/Signature

PLAGIARISM CLEARANCE PAGE

I hereby declare that all information in this document has been obtained and presented in accordance with academic rules and ethical conduct. I also declare that, as required by these rules and conduct, I have fully cited and referenced all material and results that are not original to this work.

Student First Name, Last Name:

Signature:

Abstract

User-Driven Product Analysis via Web Scraping & Multi-modal NLP: Sentiment Analysis, Feature Extraction, and Recommendation using Amazon Reviews

Tershana, Gerind.

B.Sc. in Computer Science

Thesis Advisor: Dr. Marjana Prifti Skenduli

Online retail platforms like Amazon thrive on their ability to understand and cater to customer sentiments and preferences. In this paper, we propose a novel approach for user-driven product analysis leveraging multi-modal natural language processing (NLP) techniques. Our methodology involves intelligent web scraping to collect comprehensive data from Amazon product pages. We utilize a fine-tuned DistilBERT model for sentiment analysis and TF-IDF embedding for feature extraction to enhance our recommendation system. By combining advanced NLP techniques with interactive data visualization tools, our approach transcends traditional sentiment analysis, empowering users to make informed decisions about product selection and optimization. The objective of this application pipeline is to accurately scrape all reviews, up to a maximum of 500 (the limit imposed by Amazon), as tested by Puppeteer. Subsequently, the goal is to achieve a sentiment analysis score higher than 90% for each review, including the model, correlating with customer sentiment. The application culminates with a well-structured and working pipeline[1] that the user can easily comprehend, the models and APIs communicate and share data efficiently, and the data is structured, clean and precise.

Keywords: Natural Language Processing (NLP), Sentiment Analysis, TF-IDF, Hugging Face, NLTK, Flask API, Node.js, Puppeteer, Web Scraping, React, Vite, Single Page Application (SPA)

[1] The GitHub Repository for the final version of the application can be accessed here

Dedication

This thesis is the culmination of my passion and work regarding computer science through the years. It started as a passion project from a simple idea and developed into what it is today. The journey until now has had its ups and down, the euphoria of finding solutions, the long nights of studying and the countless coffee breaks.

Firstly, I wouldn't have been here without the support of my family, the liberty they gave me to follow the passion I have about technology. I saw this especially in my mother, pushing me to pursue and don't be afraid of doing what I wanted rather than what other people would see fit for me. Thank you for never doubting me and always being my lifeline.

Secondly, I would like to thank my peers, who have always stayed with me through countless arguments, discussions, weird and funny ideas, and through a journey that none of us knew where it would lead. I am especially thankful for Florian, who provided a Stoic and knowledgeable place where I could vent or share my thoughts. I am truly grateful for your support.

Thirdly, these three years wouldn't have been the same without the guidance and helping hand of my thesis advisor, professor, and in my eyes a mentor, Dr. Marjana Prifti Skenduli. I am very thankful to have had a professor full of dedication and energy like her.

I hope you like this thesis and see the work that has been put into it. Don't be shy to show your sentiment about it as well. I will leave you with a favorite quote of mine.

> Since we cannot know all that there is to be known about anything, we ought to know a little about everything.
>
> *Blaise Pascal*

Table of Contents

Abstract **iv**

Dedication **v**

Table of Contents **vi**

1 Introduction **1**
1.1 Pipeline Description 1

2 Implementation **5**
2.1 Frontend 5
2.1.1 Structure Implementation 6
2.2 Node.js Microservice 8
2.2.1 API overview 8
2.2.2 Puppeteer implementation 9
2.3 Transformers and Embeddings 15
2.3.1 TF-IDF implementation 20
2.4 Flask microservice 22
2.4.1 Data Analysis 24

3 Conclusion **25**

References **26**

List of Figures

1 Pipeline User Scheme 1
2 Pipeline Technical Scheme 2
3 Frontend Design 5
4 Frontend Tree Structure 7
5 Node API lifecycle 9
6 Dataset cleaning and labeling 16
7 Training pipeline 19
8 TF-IDF flow 21

List of Tables

1 Puppeteer Scraping Lifecycle 11
2 Puppeteer Web Crawling Algorithm 14
3 Flask API Endpoint Algorithm 23

1 Introduction

This thesis strives to show a pipeline application of using web scraping with 2 NLP models to processes and show statistical data and measurements related to a specific Amazon product. The need to treat this problem came as the phenomenon of the flood of scammy items, unreal or fake reviews, and review payment, the process where a company plants a positive review for their product, became an unignorable problem in online shopping. To tackle some of these problems, a pipeline needed to be designed or constructed to gather and analyses such data and specify the real values of an item. The specific problem to tackle is measuring the accuracy of the product claims made by the company selling it, as well as the data presented by Amazon. Correlation on both cases will be drawn from the results to show if there is any bias from both sellers and how much data is actually shown to the user. The basic pipeline structure can be shown by the schema below[1].

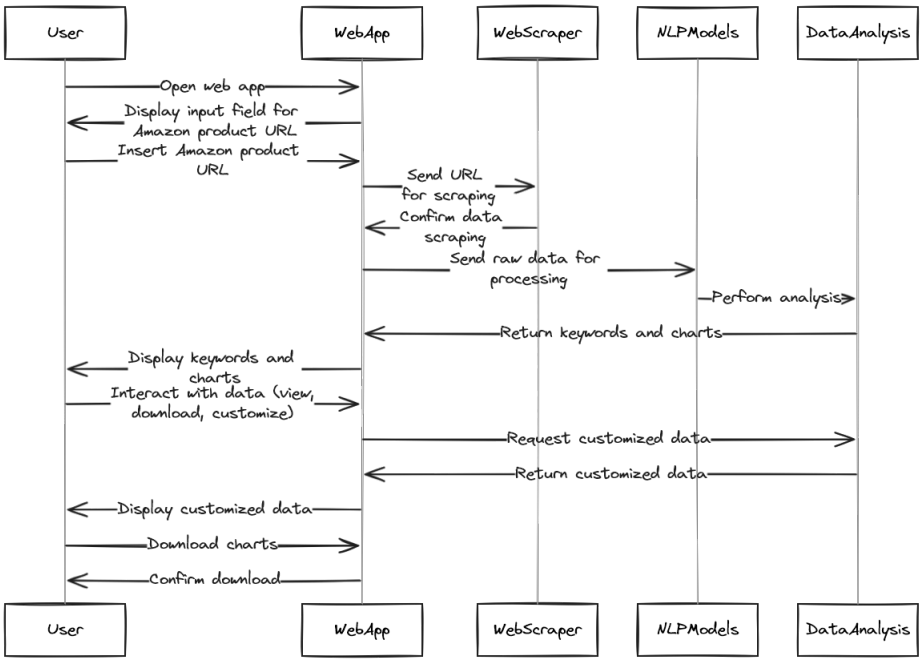


Figure 1: Pipeline User Scheme

1.1 Pipeline Description

Before going into the technologies used, it is necessary to describe the way this application or pipeline works.

The user opens the application (web application), where there they are prompted to insert a product URL from Amazon. At the moment the product URL is sent, the frontend passes it to the backend, specifically the General API, the Node.js API, which starts a web crawler to find the product and scrap all its data. After the data is scraped regarding the all the product's information, it is passed again to the Flask API, another part of the backend, which analyses and processes it through two NLP models, a semantic model build upon Distil-BER, and a TF IDF analysis to extract the keyword. Upon organizing all the data, the General API transmits it to the Frontend. The Frontend then displays the sentiment data, basic analysis such as various charts illustrating data over time or fundamental statistical analysis (excluding specific NLP analysis), the extracted keywords, and three items identified by the web crawler as closely related to the user-provided input, based on reviews and keywords.The whole system works in a serverless manner. No data about the scraping or analysis is saved, all the information is produced in real-time and shown as such.

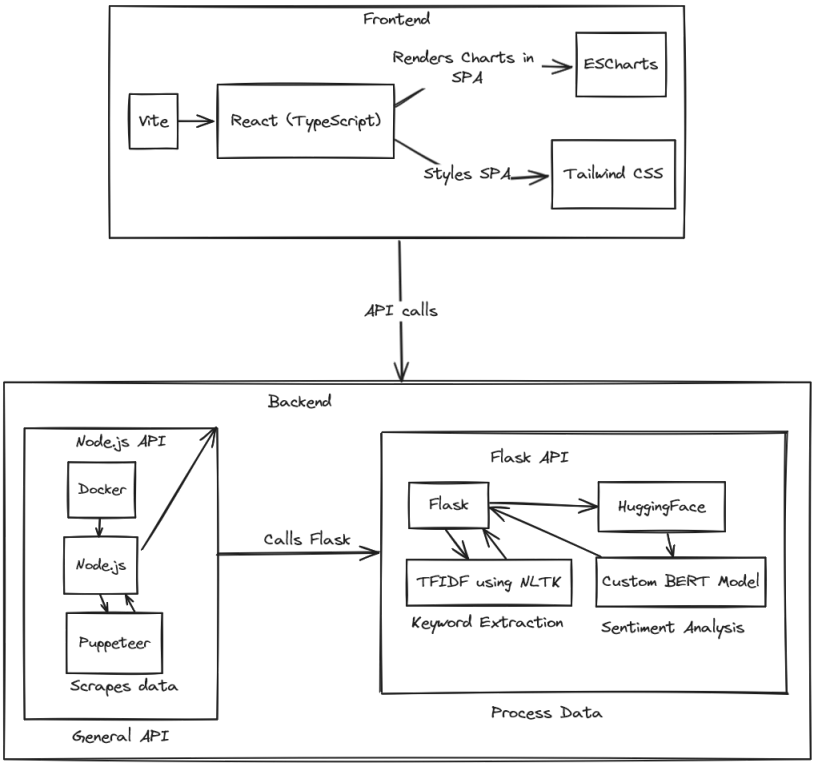


Figure 2: Pipeline Technical Scheme

In the pipeline structure[2], there are many technologies and logical connections

described. Each of them will be thoroughly discussed in the following sections, but in this section I will briefly describe what was chosen, why they were chosen and how they make the application approach the goal of the thesis.
Firstly, this is a web based application, build as an SPA (single page application). The choice to build an SPA stems from its potential to enhance accessibility for users, providing a more seamless and responsive experience. It is easier for a user to just open a web page and try a product on Amazon, than downloading a desktop app, phone app or even bootstrapping an app via CMD, or other methods. Therefore, an SPA, with easy UI/UX and straight to the point approach, makes the process more fruitful and manageable. Given that React is the most well-known and supported framework for SPAs (regarding the frontend), with countless libraries to ensure efficient pipeline functionality, it was chosen and bootstrapped by Vite. Vite is a powerful webpack alternative that enhances the performance and smoothness of SPAs, particularly when managing large amounts of data. More information regarding the implementation and the structure of the frontend will be given in the section regarding it.
Secondly, the backend is a collaboration between a Node.js API and a Flask API. The reason for two APIs instead of a single one is simple, web scraping was done with Puppeteer, a library primarily designed for Node.js. The choice for the web scraping library was a hard one and decisive, since it will primarily affect the speed and accuracy of the data that will be analyzed. As such, I choose Puppeteer for the simple reason of advance JavaScript handling when doing web scraping and as well the ability to web crawl. The choice for the Flask API comes directly for the need to run the NLP model in their original place of creation. Using the models, which were built using Hugging Face and NTLK, in a JavaScript environment would be fruitless and show an unneeded complexity, whether in time, architecture, or processing data. As such, the easiest and more efficient way was to create a small Flask API that serve the specific need. Therefore, the backend logic was divided into two small APIs, essentially playing the role of two microservices. Node.js handles the communication between the frontend, web scraping, and web crawling, managing the data exchange back and forth. The Flask API, on the other hand, is responsible for analyzing the NLP models using the data provided by Node.js from the web scraping process.
The objective of this pipeline is to achieve a multi-modal system that uses web scraping to feed a fine-tuned NLP sentiment analysis model and TF-IDF embedding to produce and analyze statistics. This will be achieved by using web scraping and web crawling powered by Puppeteer, custom API endpoints with CORS and regex for improved results. Furthermore, it will scrape all information for a product, including its description and up to 500 reviews (Amazon's limit), and to show a recommendation system it will crawl to the top 3 potential desired products for the user based on feature extraction. A Sentiment analysis model, fine-tuned from DistilBERT with a 400,000-record dataset and feature extraction based on the TF-IDF method, using the scraped data and sentiment analysis results will be the backbone for the endpoint results shown to the user. All of the above will be accumulated and shown by a Single-page application

(SPA) built with React, offering customizable settings for users to choose what to see, with graphs, statistics, and analysis displayed using Apache ECharts, with the aggregated data.

2 Implementation

2.1 Frontend

Based on the structure [2] provided until now, the pipeline uses React for the frontend. The frontend [3] will be a simple SPA that the user can interact with, to be able to see in an easy manner the data for the Amazon product they wish. React was chosen, bootstrapped by Vite, for its ease of management regarding UI/UX and data in an SPA application. UI was build on the bases of a dashboard idea. The user has two elements to interact with, the sidebar and the main section. The sidebar will be simply used for customizing the data that are shown and the way they are shown, e.g. colors. In the main section, the user should be able to see several structures.The welcome section containing what the platform is and what it provides. The help section, a small section containing a pop-up modal that shows in detail the features of the platform and how to use them. The input section, simple input field for the user to interact with, showing also fallbacks and status regarding the data that is being analyzed. The chart section, a priority section, including an amalgamation of the received data shown in charts and simple keywords so that the user can comprehend the quality of the product. Here there is also the recommended products part. In addition to the main section, the side section contains a single component that handles the changes in the way components in the main section behave. In it, the user can change the colors of the charts and the labeled data, the amount of charts that are shown, and the extra information that is shown.

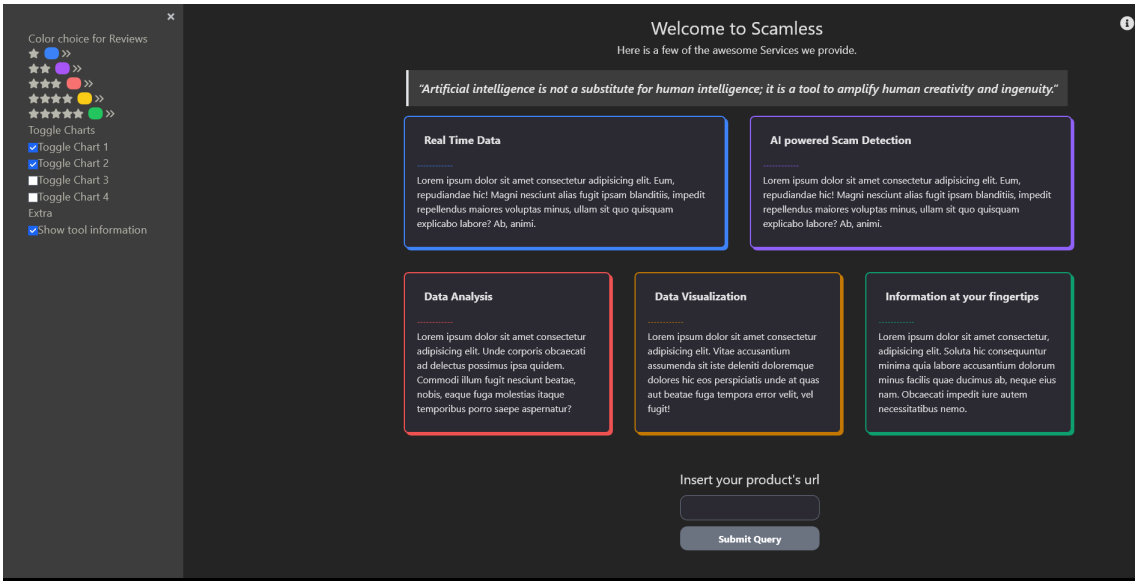


Figure 3: Frontend Design

2.1.1 Structure Implementation

React is a declarative, efficient, and flexible JavaScript library for building user interfaces. It enables developers to create interactive and dynamic UI components, which are the building blocks of modern web applications. React follows a component-based architecture, where UIs are broken down into small, reusable pieces, making development more manageable and scalable (Reddy, 2021).
In React, everything is a component. A component is a self-contained unit that encapsulates a piece of UI along with its logic. Components can be simple, like a button or a text input, or complex, like an entire form or a dashboard. They promote reusability and maintainability by allowing developers to compose UIs from smaller, modular pieces. The components that will be used in this platform are: Main Component, Side Component, Search Component, Information Cards, Recommendation Cards, Modal Popup Component, Chart Component. More information about the specifics of each component will be discussed further in this thesis.
State is a built-in feature of React that allows components to manage their own data. It represents the current condition or state of a component and determines how it should render. Stateful components have their own internal state, which can be updated in response to user actions or other events. When the state of a component changes, React automatically re-renders the component to reflect the updated state. Stateless component on the other hand are ones that just use data being passed down to them, they do not change it, or they can be rendered without data at all. These components are called dummy components. The general state pieces used and structured in this platform are several.Scraped and analyzed data, big object containing the data coming from backend, initial state compromised of dummy data until the actual data is needed. Initial layout data, state, containing information regarding the colors or placement of the data in the webpage. General input state, information regarding user interaction with the frontend. General error catching state, information regarding errors or stage of the loaded data.
To add to the whole picture, all the components are called by a parent component called App.jsx. The name is arbitrary, but the function is not. The main parent component coordinates the interaction among all the child components, providing data, specific state, and basic props to control the UI's behavior. The frontend is kept light even though it deals with a large amount of data. This happens because the frontend doesn't mutilate the data it receives but just interpret them, maintaining the original structure of a lightweight, SPA, serverless application. All of the above can also be seen in the below tree structure [4] scheme of components and the way data flows through them.

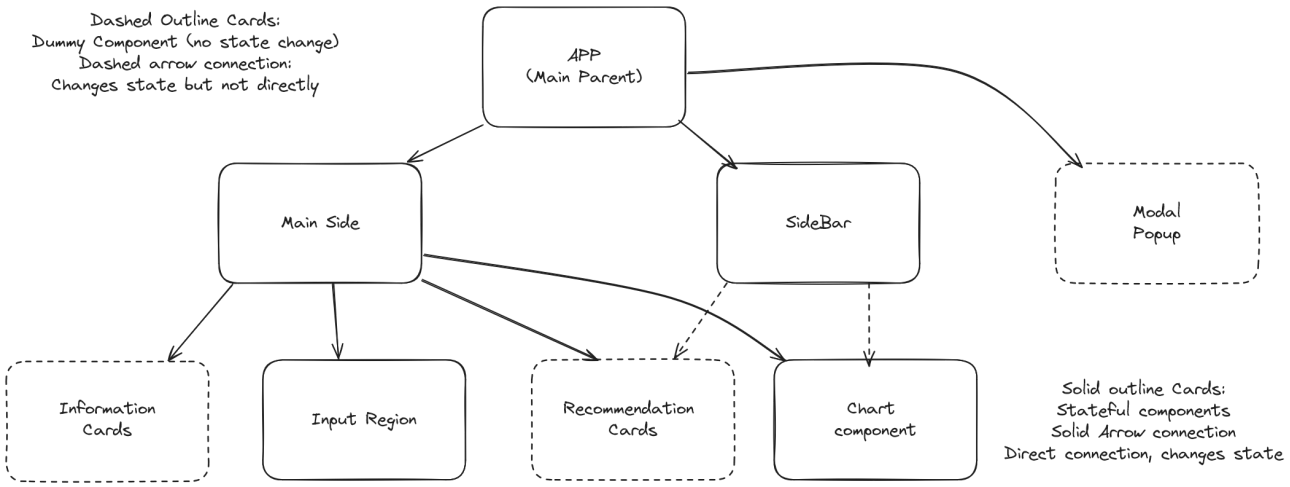


Figure 4: Frontend Tree Structure

The schema shows the way each component is connected to each other. To add context to the connections, there is a separation between dashed and solid outline elements. The dashed outline cards are components that just serve the data they are given, props passed down by the parent, but they don't mutilate them at all. These are the dummy components that were discussed above. Furthermore, the dashed outline arrows show connections between two components, that use one another, but are not directly related. These components usually use or change the state by the process of prop drilling or a state store. In this case, the usage of a state store would be redundant and would present unneeded complexity. The frontend is being build on the bases of staying light and modular, a state store would be beneficial in the modularity side, but not on the lightweights, since the app is very small to really benefit from it, making it a poor choice. As such, maintaining a prop drilling method when states or data is needed from other components was easier and more eff incent, keeping in mind that the parent component in this case is very centralized. The other side of the schema shows the solid outline. The solid outline cards are components that not only serve the data they are given, but can also mutilate them. These are the stateful components that were discussed above. Additionally, the solid outline arrows show connections between two components, that use one another, directly related. These components usually serve as another component parent, to either make another dummy component by just giving them props, or another stateful component.

The state pieces, discussed earlier, can now be given more context based on the new schema. Most of the states are being handled by the APP component, main parent, that either passes down the state with the functions to mutilate it or just as static data. The scraped data state for examples just bleeds the data from the main parent as static data to the main side component as in a waterfall

behavior. Other state pieces like the input piece are handled by the main side component, which plays the role of a parent for the input region component. Other states are handled in a similar manner to the scraped data piece, in the main parent.

2.2 Node.js Microservice

Until now the fundamentals of this pipeline have been discussed, with its objective, the overview of the technologies and the implementation of the frontend. The microservices, which were previously mentioned, are undoubtedly the backbone of this pipeline. The decision to develop two microservices for this project stems from a deliberate architectural choice aimed at achieving modularity, scalability, and maintainability (Chaplia & Klym, 2023). By separating the responsibilities of data scraping and natural language processing (NLP) into distinct microservices, the system gains several advantages. Firstly, it promotes a clear separation of concerns, allowing each microservice to focus on its specific task without becoming overly complex or tightly coupled. This modularity enhances code readability, facilitates easier testing, and simplifies future modifications or updates. Secondly, it enables scalability, as each microservice can be independently scaled based on its workload and resource demands. This ensures efficient resource utilization and helps maintain consistent performance, even under varying loads. Finally, by isolating the scraping and NLP functionalities into separate microservices, the overall system becomes more resilient to changes or updates in either component. This approach fosters flexibility and agility, allowing for easier evolution and adaptation of the system over time. In this section, it will be clarified how the General API, or in more concise terms, the Node.js API, handles the data, its general purpose and the role Puppeteer plays in all of it.

2.2.1 API overview

Node.js will not handle any rendering or serving of files. The primary purpose of Node.js is usually to provide a runtime for server rendered web applications. Such thing will not happen in this pipeline, since it is modified to play the role of a coherent, fast and lightweight API. Express.js was chosen for this purpose, since it is an industry standard for building reliable APIs using Node.js, and it provides a wide range of tools and libraries to work with. To set up a proper API, the first stepping stones are the rules of handling data. Such rules are the CORS setup, that being from which origin domain can the API communicate, receiving and sending data, helmet middleware, which provides security for different headers or header attacks. Additional middlewares are Morgan, which adds a logger for all the communication the API will make, and JSON, which, as the name implies, parses data from endpoints in JSON.
The microservice is build to serve one endpoint. At first, it seems humorous, to build a whole microservice to consume one endpoint, but there is more to that. The endpoint, defined as /api/data, calls a complex scraping function

that uses Puppeteer to scrape and analyze all the data, always keeping in mind that this is a basic data analysis. At this point there are two choices to be made, either another endpoint would be constructed for the call to the Flask API, or the call would be handled by the same endpoint just in a different time. Arguable, the better choice was the later one. When analyzing the two choices, it becomes clear why the second one performs better and why it sticks to the ideology of this pipeline. If the first choice was to be implemented, that would mean that after the scraping function would return the data, it would go to the frontend, where from there another call to the Flask API would be made. For everyone who has created APIs, this scenario proves to defy the ideology of keeping a streamlined, scalable microservice. The first approach would lead to longer load times, complexity in handling data, and more security issues since now the frontend would need to essentially communicate with two APIs. Clearly, this approach would lead to lose of modularity efficiency and present more complexity than is needed. Node.js is capable of making request, since it is a JavaScript runtime, after all. As such, the most straightforward approach, proved to be the best one. Making a request to the Flask API directly after Puppeteer has returned its data enhances security, as it maintains a direct connection between the APIs without unnecessary back-and-forth communication, and improves efficiency, since the data will be delivered to the frontend all at once upon completion.

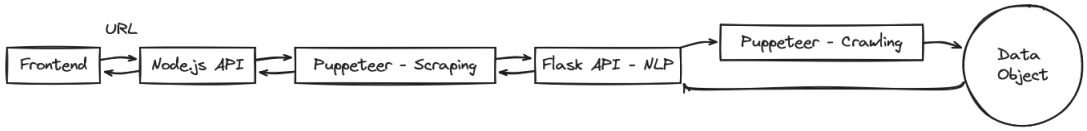


Figure 5: Node API lifecycle

To finalize the structure, the Node.js API lifecycle [5] starts with setting up Express.js and middlewares. It then invokes the custom Puppeteer function to scrape data based on the provided URL. Following that, it directly connects to the Flask API for NLP analysis. Upon receiving the analyzed data, the API processes it further, identifies the top 3 products, and sends the combined results to the frontend.Furthermore, we will discuss how Puppeteer operates, including its handling of URLs and its capabilities in crawling and scraping all necessary data.

2.2.2 Puppeteer implementation

Puppeteer is a Node.js library developed by Google that provides a high-level API to control headless Chrome or Chromium browsers. It enables developers to automate web browsing tasks, including web scraping, automated testing, and simulating user interactions with web pages. Given the context of this thesis, which involves a Node.js microservice for data scraping, Puppeteer presents several advantages over alternatives such as Cheerio and Beautiful Soup.

Firstly, Puppeteer is highly effective in handling dynamic content. It operates a full browser instance, thereby allowing it to render and interact with web pages that rely heavily on JavaScript for loading content. This capability is crucial for scraping modern web applications built with frameworks such as React, Angular, or Vue, which often load data dynamically. In contrast, Cheerio and Beautiful Soup are designed to parse static HTML and XML content and cannot execute JavaScript. Consequently, they are unsuitable for scraping websites that require JavaScript to display content.
Secondly, Puppeteer offers a rich API for browser automation. This API facilitates control over all aspects of a browser, including taking screenshots, generating PDFs, simulating user interactions (such as clicks and typing), and intercepting network requests. This comprehensive control is beneficial for complex scraping tasks that require more than merely parsing HTML. While Cheerio and Beautiful Soup provide robust tools for navigating and manipulating the DOM, they lack the ability to interact with a web page in real time.
Thirdly, Puppeteer supports scalability and modularity within the microservice architecture. It integrates seamlessly with Node.js, leveraging Node.js's non-blocking, event-driven nature for efficient data scraping. This alignment supports the modularity and scalability objectives of this project, allowing the scraping microservice to be independently scaled based on workload. In contrast, Cheerio and Beautiful Soup are not Node.js libraries—Cheerio is compatible with Node.js, but Beautiful Soup is a Python library. Using a Python-based scraping solution like Beautiful Soup could introduce additional complexity and overhead to the system.
Lastly, Puppeteer ensures direct integration with Node.js. As a Node.js library, Puppeteer naturally fits into the Node.js microservice, simplifying both development and deployment processes. This seamless integration promotes better performance and easier maintenance. Although Cheerio is also a Node.js library, it lacks the capability to handle dynamic content rendered by JavaScript, limiting its effectiveness for modern web scraping needs. Beautiful Soup, requiring a separate Python environment, adds complexity to the deployment and necessitates communication between the Node.js microservice and a potential Python scraping tool.
The first thing to be implemented with the usage of Puppeteer was the scraping of data [1]. This is a complex system since there are some limitations. The first problem to be encountered was the limitation of Amazon. If you browse as a normal user for a specific product on Amazon, you can go for at most 500 reviews, that being 100 reviews for each type e.g. one star. Thus, since scraping is the method of automating a user behavior, getting more than 500 reviews for the analysis would be impossible as of writing this thesis. The second problem is being detected by Amazon. Being detected as an automatic scraping bot is expected behavior from this pipeline. Cybersecurity has evolved through the years, with Cloudflare, captchas and different mechanics to prove that the one using the browser is an actual person and not a crawler. As such, this problem can only be bypassed to a certain point, but not be solved. The object of this thesis is to provide a working pipeline that uses web scraping, but not to ex-

ploit the architecture of Amazon Web Services. The bypassing was done by the plugins of Puppeteer, being stealth puppeteer and headless puppeteer, which makes the putter instance be lightweight, without cached data, and incognito each time it scrapes or crawls. The following pseudocode [1] shows an overview of the steps taken by the scraping instance of puppeteer.

Table 1: Puppeteer Scraping Lifecycle

Function initializeBrowser

```
function initializeBrowser
    Load Stealth Plugin for Puppeteer
    Set custom arguments for Puppeteer
    Launch Puppeteer browser instance with custom arguments and plugins
    return browser instance
end function
```

Function deriveReviewsURL

```
function deriveReviewsURL(productURL)
    Extract product ID from the URL using regex
    Construct reviews URL using the extracted product ID
    return reviews URL
end function
```

Function scrapeProductDetails

```
function scrapeProductDetails(page, url)
    Navigate to the product page
    Wait until DOM content is loaded
    Initialize data structure to store product details
    Scrape product price and store in data
    Scrape product title and store in data
    Scrape product details section and store in data
    Scrape about item information and store in data
    Scrape general product details and store in data
    Scrape general description items and store in data
    return data
end function
```

Function scrapeReviewsByRating

```
function scrapeReviewsByRating(page, url, filterRating)
    Construct the reviews URL with the specific rating filter
    Navigate to the reviews page
    Wait until DOM content is loaded
    Initialize data structure to store reviews
    pageNum ← 1
    while more review pages exist do
        Extract reviews from the current page
        for each review do
            Scrape author, review text, rating, rating description, and time
            Store scraped review data in structured format
        end for
        Navigate to the next page if it exists
        pageNum++
        Wait for page to load
    end while
    return reviews data
end function
```

Function performDescriptiveAnalysis

```
function performDescriptiveAnalysis(dataset)
    for each product in dataset do
        for each year in product reviews do
            Calculate mean, median, standard deviation, and range of
            review counts
            Store descriptive analysis results
        end for
    end for
end function
```

Function runCrawler

```
1: function runCrawler(urls)
2:     Initialize dataset to store all scraped data
3:     starRatings ← ["five_star", "four_star", "three_star", "two_star",
       "one_star"]
4:     for each url in urls do
5:         browser ← initializeBrowser
6:         page ← browser.newPage()
7:         productDetails ← scrapeProductDetails(page, url)
8:         reviewsByRating ← {}
9:         for each rating in starRatings do
10:            reviews ← scrapeReviewsByRating(page, url, rating)
11:            reviewsByRating[rating] ← reviews
12:        end for
13:        Combine productDetails and reviewsByRating into a single product
           dataset
14:        Add product dataset to the main dataset
15:        browser.close()
16:    end for
17:    performDescriptiveAnalysis(dataset)
18:    return dataset
19: end function
```

The implementation shows the correlation between the crawling, scraping and data analysis done by puppeteer. It might seem complex, but all is managed by the main function called runCrawler [1]. The function starts firstly formats the URL, to point to the product's review page, then starts two similar scraping functions, both using the previously mention plugins of puppeteer. It firstly scrapes the basic data for the product being its description details and price, and then passes on each review section, from five stars to one star, scraping author, review text, rating, rating description, and time. All of this data is appended to the big JavaScript object called dataset. Again, the name is arbitrary, but the function is not. The object will go through a descriptive analysis, adding more keys, and then properly formatted for the return to the API. After all of this lifecycle ends, the instance of the browser will be terminated, the data will be returned to the API endpoint and then be sent to the Flask API for further analysis. This is where the mainline scraping from puppeteer ends. The process shows to be fast and efficient, taking roughly two minutes on large reviews and less than a minute on low reviews. This performance can drastically change based on physical resources or network architecture.
The other instance, when puppeteer is used, is on the crawling [2] function using

the keyword given by the Flask API and basic information previously scraped by the first instance. The crawling uses the same plugins and setup as the first usage of puppeteer, but here it specifically shows the strength puppeteer provides, the accessibility of generating input in HTML field and masking the automatic process as a user, where in this case we are trying to find the top three products. The following pseudocode [2] shows how the crawling is implemented.

Table 2: Puppeteer Web Crawling Algorithm

Function searchAmazon

```
function searchAmazon(keyword, minPrice, maxPrice)
    Initialize browser instance
    Navigate to the Amazon homepage
    Input the search keyword into the search bar
    Click on the search button
    Wait for the search results to load
    Extract product items from the search results
    Filter product items based on price range
    Scrape product details such as name, price, image, URL, and rating
    Store scraped product data
    Close the browser instance
    return scraped products
end function
```

Function runSearchCrawler

```
function runSearchCrawler(searchTerm, minPrice, maxPrice)
    Set search keyword, minimum price, and maximum price
    Call searchAmazon function with search parameters
    Display scraped products
end function
```

The crawling takes as input the searchTerm, minimum price and maximum price. The search term is a string being concatenated by the keywords inferred from the models and the details previously scraped, to find the best possible product description. This is inputted by the puppeteer agent, and when the results appear it goes through them to find the one that closely relates to the given user product, based on a simple review filter and price filter. The returned products will have information regarding their name, price, image, URL, and rating, which will be appended to the object that will be returned to the fronted. Such data is needed to construct a proper visual cue for the user. This is the final purpose of puppeteer, leading to the closure of its lifecycle with the data returned from the API endpoint.

2.3 Transformers and Embeddings

The backbone of the data analysis is the usage of the transformers and embeddings the pipeline contains. The idea of multimodal NLP is backed up by two architectures, the fine-tuned DistilBERT model and the TF-IDF custom embeddings. One of the needs of the pipeline was to understand the feelings of the users from the reviews that they have left and extract the main ideas they have said. Such behavior is perfectly contained by the chosen architectures. The choice for DistilBERT stems from its efficiency in space and complexity. DistilBERT, a distilled version of BERT (Bidirectional Encoder Representations from Transformers), is a state-of-the-art model designed for natural language processing (NLP) tasks. DistilBERT retains 97% of BERT's language understanding while being 60% faster and 40% smaller, making it an optimal choice for sentiment analysis, especially when review length text (Sanh et al., 2020). On the other hand, Term Frequency-Inverse Document Frequency (TF-IDF) is a statistical method used to evaluate the importance of a word in a document relative to a corpus (Robertson, 2004). TF-IDF is effective for keyword extraction because it helps to identify words that are both frequent in a particular document and distinctive across a larger collection of documents. It looks closer to an embedding than a full-fledged AI model, but this works to the pipelines favor, of containing modularity, scalability and efficiency. The dataset powering the fine-tuned model is an Amazon review based dataset, spanning a period of 18 years, including 35 million reviews up to March 2013 (Zhang et al., 2016). Reviews include product and user information, ratings, and a plaintext review. This dataset was taken from the Stanford Network Analysis Project (SNAP), and then cleaned and labeled. We will dive deeper on how the model is trained, the data is prepared and the embeddings is constructed. The requirements for a proper fine-tuned model, especially a fine-tuned DistilBERT model for sentimental analysis, compose of a proper labeled and cleaned dataset, oriented in quality than in quantity, and a training script for the model. The dataset as specified earlier spans from 18 years and is a large one. DistilBERT is an efficient model, therefor the need for the full dataset was irrelevant. As such, the first modification to the dataset was to shrink it to a proper size, that being 400'000 records. The data is formatted in parquet format and uses the Hugging face dataset loader for the modification and reading of data.

The preprocessing of the Amazon dataset involves several crucial steps to clean the data and prepare it for sentiment analysis. We need to describe the formal procedure for installing necessary dependencies, setting up the sentiment analysis pipeline, and cleaning and labeling the dataset. The process ensures that the data is in an optimal format for model training and evaluation. To begin with, the required dependencies are installed. These include libraries such as datasets, pandas, transformers, and torch, which are essential for handling the dataset, performing sentiment analysis, and leveraging GPU acceleration.A sentiment analysis pipeline is set up using the transformers library from Hugging Face, leveraging a pre-trained DistilBERT model. The device is set to utilize a GPU, to speed up the computation. The pipeline utilizes the model

nlptown/bert-base-multilingual-uncased-sentiment, which is specifically trained for multilingual sentiment analysis. This model processes the text input and predicts sentiment scores across multiple languages. The choice for the pre-trained BERT model for labeling, stem from it being the standard in the basic pipeline that Hugging Face or Kaggle uses regarding semantic values. After the loading of the dataset, initially the label column is removed, and a new text column is created by concatenating the title and content columns. This step ensures that the textual information is consolidated into a single column for efficient processing. The following schematic representation [6] outlines the steps involved in cleaning and preparing the test dataset:

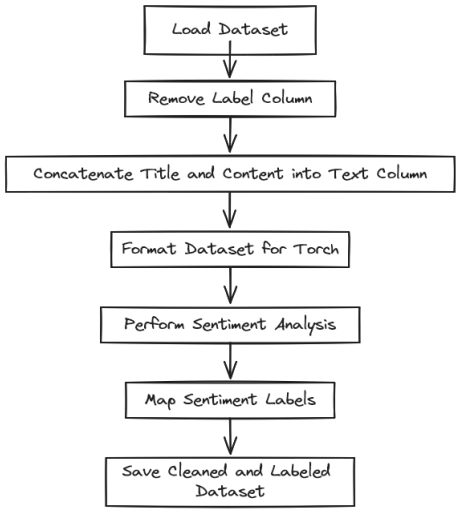


Figure 6: Dataset cleaning and labeling

The dataset is then formatted for PyTorch, facilitating efficient data handling during the sentiment analysis process. A custom mapping function is applied to each example in the dataset to perform sentiment analysis using the pre-trained model, and the results are appended to a list. This function processes the text in batches, with a specified batch size of 4, ensuring that the pipeline can handle large datasets efficiently without overloading the computational resources. The results are appended to a list for further processing. A function is defined to convert the model's sentiment predictions into more intuitive labels. The sentiments are categorized as "Perfect," "Satisfied," "Neutral," or "Negative" based on the star ratings. This step transforms the raw model outputs into user-friendly labels. The sentiment results are mapped back to the dataset, creating new columns for the sentiment labels and their associated scores. This enriched dataset is then converted to a pandas DataFrame and saved to a CSV file for future analysis. The same procedure is repeated for the training dataset. The training data is loaded, cleaned by removing the "label" column, and a new "text" column is created. Sentiment analysis is performed on the cleaned

text, and the results are mapped back to the dataset. The labeled dataset is converted to a pandas DataFrame and saved.
This formalized procedure ensures that both the training and test datasets are thoroughly cleaned and labeled, facilitating accurate sentiment analysis. The use of a pre-trained BERT model, combined with effective data preprocessing and labeling, enhances the quality and usability of the dataset for sentiment analysis tasks. The entire process is depicted in the schematic representation above, highlighting the key steps involved in preparing the dataset for analysis. The process proves to be efficient, seen by the time it took for all the computation to be finished. When a small batch size and CPU was used for the calculations, the dataset labeling would take five hours for 360'000 records, whereas with the usage of a GPU and the proper batches the time was reduced to one hour and forty-two minutes.
The process of fine-tuning a DistilBERT model for sentiment analysis on a subset of the Amazon Polarity dataset involves several crucial steps, each aimed at preparing the data and optimizing the model's performance. This section [7] details the dataset preparation, tokenization, model training, and evaluation, providing a comprehensive overview of the methodology.
To begin, the dataset is loaded from the Hugging Face repository using the datasets library. The subset, GerindT/mini_amazon_sentimental, is split into training and testing sets, with 20% reserved for testing. The dataset is then cleaned to retain only the essential information by removing the columns "title," "content," "label," and "score." The "sentiment" column is renamed to "label" to standardize the nomenclature for training. This is the dataset that was labeled and deployed into hugging face.The sentiment values in the dataset are mapped to numerical labels to facilitate model training. This mapping assigns integers to sentiment categories: Perfect is mapped to 3, Neutral to 1, Negative to 0, and Satisfied to 2. This conversion ensures compatibility with the classification model, which requires numerical inputs for the label column.The next step involves tokenizing the dataset using the DistilBERT tokenizer. The AutoTokenizer from the transformers library is utilized to load the distilbert-base-uncased tokenizer. Tokenization is performed with truncation enabled, ensuring that all input sequences fit within the model's maximum length constraints. The tokenizer function processes the text data into tokens that the DistilBERT model can understand.For the model training, the AutoModelForSequenceClassification class from the transformers library is employed. This class is specifically designed for sequence classification tasks and is instantiated with the DistilBERT base model. The model is configured to handle four sentiment labels corresponding to the numerical labels defined earlier. It is loaded onto the appropriate device, utilizing a GPU if available, otherwise defaulting to the CPU. This ensures efficient computation during the training process.Training arguments are set using the TrainingArguments class from the transformers library. Key parameters include the number of training epochs, batch size for training and evaluation, weight decay, and strategies for evaluation and saving the model. Specifically, the training is configured for two epochs with a batch size of 16 for both training and evaluation. The evaluation

strategy is set to epoch, meaning evaluation occurs at the end of each epoch, and the best model is saved based on performance metrics. The Trainer class is then used to manage the training and evaluation processes. The trainer is initialized with the model, training arguments, dataset, tokenizer, and a custom function for computing metrics. The compute_metrics function uses the accuracy metric from the evaluate library, comparing the predicted labels to the true labels to assess model performance. Training the model involves calling the trainer.train() method, which iterates over the dataset for the specified number of epochs, optimizing the model's parameters. Following training, the model is evaluated on the test dataset to measure its accuracy and other performance metrics. Upon completion of training and evaluation, the model is pushed to the Hugging Face Hub, allowing it to be accessed and shared. This is done using the trainer.push_to_hub() method, which uploads the trained model and its configurations.

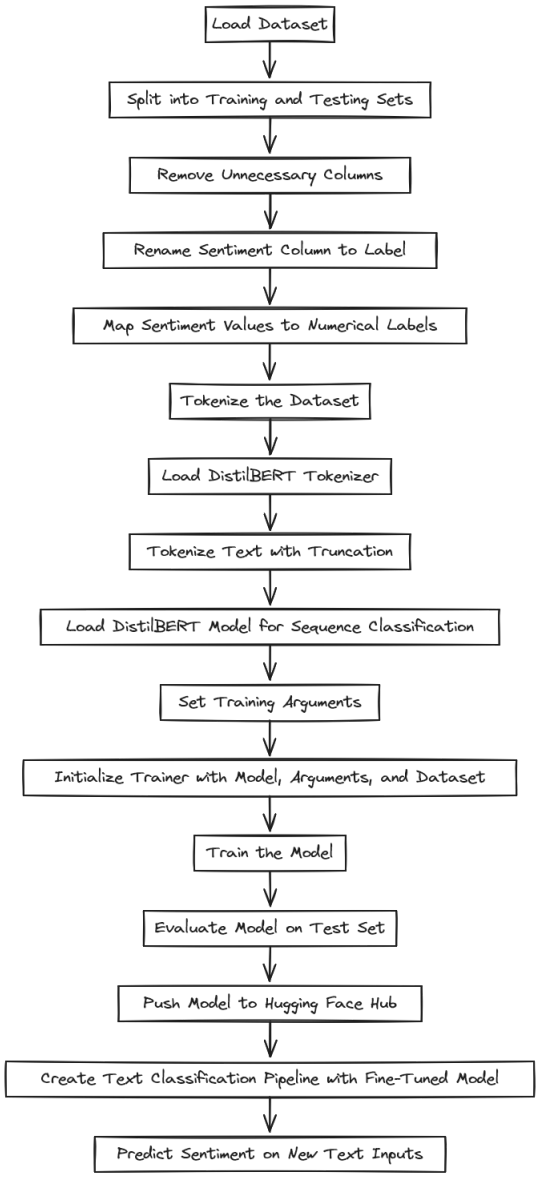


Figure 7: Training pipeline

For model predictions, the fine-tuned model is loaded using the pipeline function from the transformers library, creating a text classification pipeline. This pipeline is then used to classify new text inputs, providing sentiment predictions based on the fine-tuned model. The training for two epochs took approximately two hours, using the training dataset and leveraging the performance boost CUDA cores and parallelization from a GPU. The final metrics of the training

show an evaluation accuracy of 0.92 and a training loss of 0.25.

2.3.1 TF-IDF implementation

In the context of natural language processing, the Term Frequency-Inverse Document Frequency (TF-IDF) method is utilized for extracting meaningful keywords from text data. This approach is particularly useful in sentiment analysis to highlight significant terms that contribute to understanding user sentiments. The following outlines the implementation of TF-IDF using Python's Scikit-learn library and the Natural Language Toolkit (NLTK) for preprocessing text. First, essential libraries for TF-IDF vectorization and text preprocessing are imported, including tools for tokenization, stop words removal, and lemmatization.Next, necessary NLTK resources such as stop words, tokenizers, and lemmatizers are downloaded to ensure the required tools for text preprocessing are available. Stop words, which are common words that do not contribute significant meaning, are loaded, and a lemmatizer is initialized to reduce words to their base or root form, aiding in text normalization. A custom tokenizer function is defined to process the input text. This function performs several steps: tokenization, lemmatization, filtering non-alphabetic tokens and words shorter than three characters, and removing stop words. These preprocessing steps ensure that the text is clean and standardized for TF-IDF analysis. To illustrate the process formally, the following schematic representation [8] outlines the steps involved:

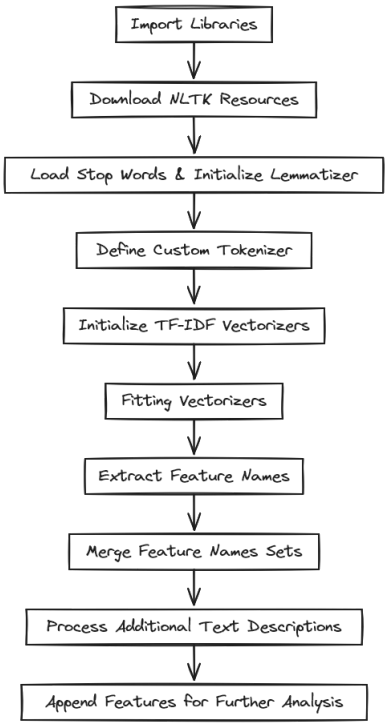


Figure 8: TF-IDF flow

The TF-IDF vectorizers are then initialized with the custom tokenizer and specific parameters. These parameters include setting a minimum document frequency (min_df=3) to ignore terms appearing in fewer than three documents, and a maximum document frequency (max_df=0.8) to exclude terms found in more than 80% of the documents. The number of features is limited (max_features=10) to focus on the most significant terms. The vectorizers are fitted to the review text arrays, computing the TF-IDF scores for the terms in the documents. Following this, feature names (keywords) are extracted from each vectorizer, and the unique keywords from both sets are merged to create a comprehensive set of features. Additionally, text descriptions from the dataset are processed using similar TF-IDF vectorizers. Each description is vectorized, with parameters set to ensure that all terms are considered, allowing for detailed keyword extraction from individual descriptions. The extracted features are collected for further analysis. This implementation showcases an efficient approach to constructing and utilizing TF-IDF for keyword extraction in sentiment analysis. By employing custom tokenization and preprocessing steps, the model ensures that the most relevant and significant terms are identified from the text.

2.4 Flask microservice

The decision to employ Flask for the API component of the backend was influenced by several pivotal factors. Foremost, the NLP models utilized in the project, constructed using Hugging Face and NLTK, are fundamentally Python-based. Executing these models within their native Python environment through Flask simplifies the integration process, circumventing the complexities that would arise from attempting to port these models to a JavaScript environment. Additionally, Flask's lightweight nature and minimalistic design render it an optimal choice for creating small, efficient APIs (Aslam et al., 2015). This aligns with the project's objective of maintaining a modular and maintainable backend structure. Furthermore, the architectural design benefits significantly from a direct communication approach between the Node.js and Flask APIs. This allows data, once scraped by Puppeteer through the Node.js API, to be seamlessly passed to the Flask API for NLP analysis. Such a streamlined process reduces latency and enhances security by minimizing the need for the frontend to interact with multiple APIs.

Flask, a micro web framework written in Python, is designed for simplicity and extensibility, making it particularly suitable for this application. Additionally, it supports middleware integration, allowing for functionalities such as logging, security, and data parsing to be incorporated effortlessly. The framework's extensibility allows it to be augmented with various libraries, enhancing functionalities such as AI model integration. The integrations done to the Flask API are similar to the ones done to the Node.js one. Firstly, it uses the build in HTTP support of flask, with the ability to handle request and JSON datatype, which will be the way flask will communicate with Node.js. Additionally, it is needed to implement the CORS rules, in the same fashion as in the Node.js structure, to enhance the reliability and security of the microservice. Most of these libraries are build into flask, different from Node.js, but the need for third party libraries arose as well in this microservice since it will handle the analysis of AI models. The AI model for sentimental analysis was trained and deployed using the Hugging Face service, as it is discussed in the previous section. As such, flask being build on top of python can use the hugging face library to implement the features of the model. The other library it uses are the TF IDF ones, that will be used in the same manner as in vanilla python. Since the way these models were built is discussed in the previous section, below [3] will be described how the models are used, how the data is analyzed, cleaned and appended to the JSON object that will be returned to the pipeline.

Table 3: Flask API Endpoint Algorithm

API Endpoint Analysis

```
1: function API
2:     Get JSON data from the request body
3:     Extract total reviews from the request data
4:     Initialize analyzed_reviews as an empty list
5:     for each review in total_reviews do
6:         Extract title and text from the review
7:         Check if title or text is None, set to empty string if so
8:         Concatenate title and text to form review_text
9:         Classify review_text using sentiment classifier
10:        Add classification score and sentiment label to the review
11:        Append the review to analyzed_reviews
12:    end for
13:    Extract sentiments from analyzed_reviews
14:    Count occurrences of each sentiment and store in sentiment_results
15:    Filter reviews with non-empty reviewText into reviews_with_text
16:    Separate reviews into reviews_label_01 and reviews_label_23
       based on sentiment
17:    Initialize and fit TF-IDF vectorizes for reviews_label_01 and
       reviews_label_23
18:    Extract feature names from both vectorizes and merge them into
       merged_feature_names
19:    Initialize description_features as an empty list
20:    for each description field in request data do
21:        if description field is not None and not empty then
22:            Initialize and fit a TF-IDF vectorizer on the description field
23:            Append feature names to description_features
24:        end if
25:    end for
26:    for each product detail in request data do
27:        Append detail values to description_features
28:    end for
29:    Flatten the description_features list
30:    Update request data with analyzed reviews, sentiment results,
       keywords, and description features
31:    return JSON response with updated request data
32: end function
```

2.4.1 Data Analysis

The process [3] begins with the receipt of JSON data from the request body, specifically extracting the totalreviews key, which contains the customer reviews to be analyzed. Each review undergoes a thorough sentiment analysis using the pre-trained sentiment classifier built before. The classifier processes the concatenated title and text of each review, determining the sentiment and assigning a corresponding score. These results are appended to an analyzed_reviews list. Following the sentiment analysis, the system aggregates sentiment data across all reviews, categorizing them into predefined labels: negative, neutral, positive, and very positive. This aggregation helps in generating a sentiment summary, which is stored in the sentiment_results dictionary. Additionally, reviews are filtered to exclude those with empty reviewText fields, ensuring that only meaningful data is processed further.
The reviews are then divided based on their sentiment labels into two categories: reviews_label_01, containing negative and neutral sentiments, and reviews_label_23, containing positive and very positive sentiments. For each category, a Term Frequency-Inverse Document Frequency (TF-IDF) vectorizer is initialized and fitted, with a custom tokenizer applied to extract significant features from the review texts. The feature names extracted by the vectorizers are merged into a comprehensive set, representing the key terms across all reviews. This is the same TF-IDF embedding that was build in the previous section. Moreover, the microservice processes additional product description fields provided in the request data. Each description field undergoes similar TF-IDF vectorization to extract relevant keywords, which are then compiled into a description_features list. This list is flattened to ensure a consistent format for subsequent processing.
Finally, the microservice updates the request data with the analyzed reviews, the sentiment summary, the merged keyword features, and the description features. This enriched data is returned as a JSON response, facilitating further integration into the broader data pipeline. The structured approach in handling and analyzing the data ensures a high level of accuracy and relevance in the insights generated, contributing to a more informed decision-making process in the application's backend operations. The use of Flask proves satisfactory, as the pipeline executes API endpoint classifications for reviews of varying lengths in less than a minute.

3 Conclusion

This thesis has successfully demonstrated the development and implementation of a comprehensive pipeline for user-driven product analysis using web scraping and multi-modal natural language processing (NLP) techniques. The pipeline integrates various technologies and methodologies to address the prevalent issue of fake reviews and misleading product information on online retail platforms like Amazon. By leveraging intelligent web scraping and advanced NLP models, such as DistilBERT for sentiment analysis and TF-IDF for feature extraction, the system provides a robust mechanism for extracting and analyzing product data.

The results of this pipeline highlight the effectiveness of combining web scraping with NLP models to process and present statistical data and measurements related to specific Amazon products. The pipeline's ability to accurately scrape reviews and perform high-accuracy sentiment analysis empowers users to make more informed decisions based on real and unbiased product reviews. The sentiment analysis achieved an accuracy of 92.32% with a training loss of 0.2505 after two epochs of training using a GPU. Additionally, the real-time nature of the pipeline ensures that users receive up-to-date and relevant information without the need for data storage, enhancing privacy and efficiency. The entire process of scraping and analysis takes approximately two minutes for large reviews and less than a minute for smaller ones.

This project not only addresses the immediate problem of fake reviews but also sets a foundation for future enhancements. Potential improvements could include expanding the range of products analyzed, incorporating more sophisticated machine learning models, and refining the data visualization tools to provide even more actionable insights.

To conclude, the successful implementation of this pipeline demonstrates the potential of combining web technologies and NLP in tackling real-world problems in e-commerce, paving the way for more transparent and reliable online shopping experiences.

References

Aslam, F., Mohammed, H., & Lokhande, P. (2015). Efficient way of web development using python and flask. *International Journal of Advanced Research in Computer Science*, *6*.

Chaplia, O., & Klym, H. (2023). Node.js project architecture with shared dependencies for microservices. *Measuring Equipment and Metrology*, *84*, 53–58. https://doi.org/10.23939/istcmtm2023.03.053

Reddy, M. (2021). Analysis of component libraries for react js. *IARJSET*, *8*, 43–46. https://doi.org/10.17148/IARJSET.2021.8607

Robertson, S. (2004). Understanding inverse document frequency: On theoretical arguments for idf. *Journal of Documentation - J DOC*, *60*, 503–520. https://doi.org/10.1108/00220410410560582

Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2020). Distilbert, a distilled version of bert: Smaller, faster, cheaper and lighter.

Zhang, X., Zhao, J., & LeCun, Y. (2016). Character-level convolutional networks for text classification.